SSE vs Long Polling vs WebSockets

choosing how the server pushes updates

Watch the 4-min video

> **In one line:** short polling asks on a timer, long polling holds each request until there is news, server-sent events keep one HTTP response open and stream events down it, and WebSockets give a two-way channel. Pick the simplest one that fits the direction of your data.

When you chat with an AI assistant, the answer appears word by word. That is usually **server-sent events (SSE)**: one HTTP response that stays open while the server keeps pushing tokens. Here is how it compares with polling, long polling and WebSockets.

1 Short polling and long polling

Short polling is the simplest: the client asks every few seconds, “anything new?”. Most answers are empty, which wastes requests, and an update can wait up to a full interval before anyone sees it (on average half of it).

Long polling is smarter. The client sends a request and the server **holds it open** until there is news, or until a timeout (say 30 seconds), then answers. As soon as a response arrives, the client sends the next request. Updates arrive almost instantly, and it works through any HTTP infrastructure, which makes it a great **fallback**. The cost: every message needs a full request and response, with headers, and there is a small gap while the next request is on its way.

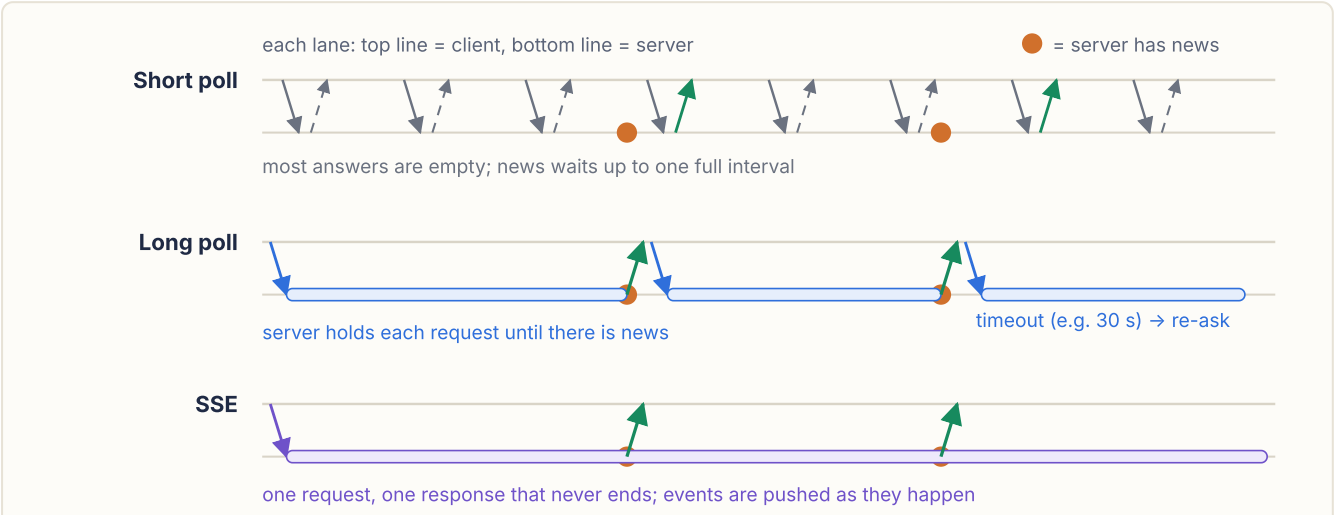


Figure 1. Three ways to get the two news events (orange) to the client. Short polling finds them late and asks many empty questions; long polling and SSE deliver them almost at once.

2 Server-sent events

With **SSE** the browser opens one request with the `EventSource` API. The server replies with `Content-Type: text/event-stream` and **never closes** the response. It writes events as they happen, as plain text: lines starting with `data:`, with a **blank line** ending each event.

```
id: 42
event: score
data: {"home": 2, "away": 0}

: keep-alive (a comment line, ignored)
```

Each event can carry an `id`. If the connection drops, the browser **reconnects automatically** (after a few seconds, or the `retry:` value the server set) and sends a `Last-Event-ID` header, so the server can **resume** from where it stopped.

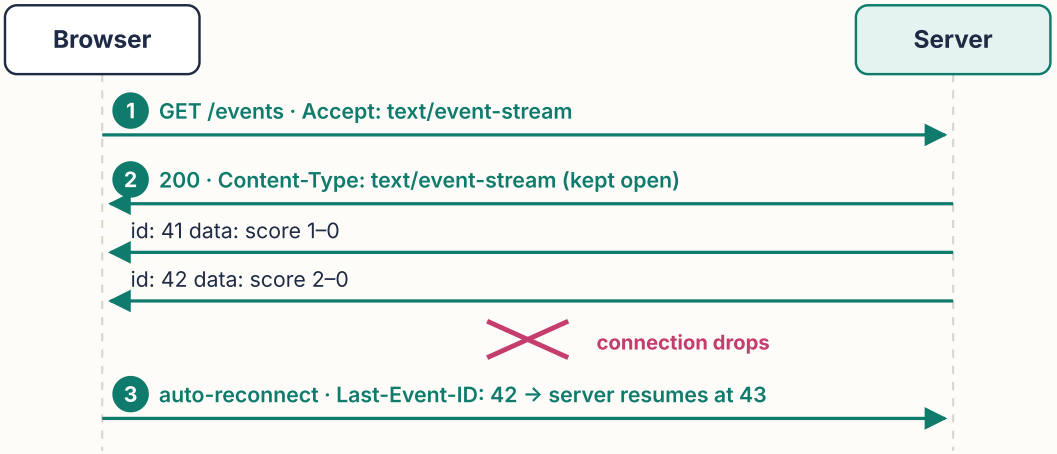


Figure 2. SSE recovery is built in: the browser reconnects by itself and tells the server the last event it saw. With WebSockets you write this logic yourself.

3 Example: streaming LLM tokens from FastAPI

The endpoint returns a `StreamingResponse` with the event-stream media type. An **async generator** yields each token as a `data:` line the moment the model produces it.

```
import json
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
app = FastAPI()

@app.get("/chat/stream")
async def chat_stream(q: str):
    async def events():
        async for token in llm.stream(q):   # your model client
            yield f"data: {json.dumps(token)}\n\n"
        yield "data: [DONE]\n\n"
    return StreamingResponse(events(), media_type="text/event-stream",
                             headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"})
```

The user sees the first words in a few hundred milliseconds instead of waiting ten seconds for the full answer. JSON-encoding each token keeps newlines inside a token from breaking the format, and `X-Accel-Buffering: no` stops nginx from buffering the stream.

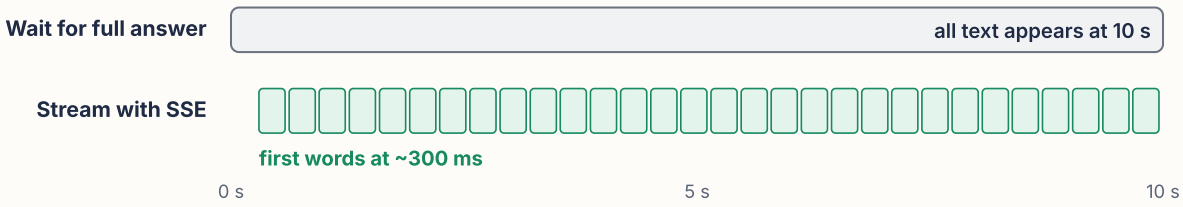


Figure 3. Same model, same total time. Streaming changes when the user starts reading: time to first token, not time to last token, is what feels fast.

4 How they compare

	Short polling	Long polling	SSE	WebSockets
Direction	Client pulls	Client pulls, server waits	Server → client	Both ways
Latency	Up to one interval	Near real time	Real time	Real time
Cost per message	Full request, often empty	Full request	A few bytes	2–14 bytes
Protocol	Plain HTTP	Plain HTTP	Plain HTTP	Upgrade to WebSocket
Reconnect	n/a	Next request	Built in, with Last-Event-ID	You build it
Infrastructure	Anything	Anything (long timeouts)	Anything that doesn't buffer	Upgrade-aware LBs, long timeouts

5 Choosing

- **Choose SSE** when updates flow mainly from server to client: notifications, live scores, dashboards, job progress and AI token streams. It is plain HTTP, so it works with HTTP/2, proxies, CDNs and your normal cookie or header authentication.
- **Choose WebSockets** when the client also sends frequent messages: chat, collaborative editing, multiplayer games.
- **Keep long polling** as a fallback for networks or proxies that break streaming, and short polling for rare, non-urgent checks.

CAVEAT: THE SIX-CONNECTION LIMIT

Over HTTP/1.1, browsers allow only about **six connections per host**, shared by all tabs. Each open EventSource uses one, so six tabs of your dashboard can freeze every other request to that host. Serve SSE over **HTTP/2**, where each stream is just one of ~100 multiplexed streams on a single connection.

WORKED EXAMPLE

A dashboard with 10,000 open users, and about 1 update per minute each. Short polling every 5 s: 10,000 / 5 = **2,000 requests/s**, about 98% of them empty, and updates arrive 2.5 s late on average. SSE: 10,000 idle connections and about 10,000 / 60 ≈ **170 small events/s**, delivered at once. Same features, roughly a tenth of the traffic.

COMMON MISTAKES

- A proxy or gzip layer buffering the stream, so events arrive in one lump at the end.
- Serving SSE over HTTP/1.1 and hitting the six-connection limit across tabs.
- Picking WebSockets for one-way updates and then building reconnection and auth by hand.

6 Key terms

Short polling: ask on a fixed interval

Long polling: server holds the request until news or timeout

SSE: one long HTTP response carrying events

EventSource: browser API for SSE

text/event-stream: SSE content type

Last-Event-ID: header sent on reconnect, to resume

StreamingResponse: FastAPI response fed by a generator

Time to first token: delay before the first words appear

Fallback: simpler transport used when the best one fails

7 Interview questions

Q1. When would you choose SSE over WebSockets? BASIC

When data flows one way, server to client, and you want plain HTTP, automatic reconnects with Last-Event-ID and simple infrastructure: notifications, dashboards, streaming LLM tokens.

Q2. How does long polling work? BASIC

The client sends a request that the server holds open until there is data or a timeout (e.g. 30 s). Then the client immediately sends another. Near real-time, works through any HTTP infrastructure, but costs a full request per message.

Q3. Design token streaming for an AI chat product. SCENARIO

POST the prompt, return a text/event-stream response, and yield each token as a data: line from an async generator; send a final [DONE] event. Disable proxy buffering, serve over HTTP/2, send keep-alive comments every ~15 s, and cancel generation if the client disconnects.

Q4. Users say live scores arrive all at once at the end. What's wrong? SCENARIO

Something buffers the response: nginx proxy_buffering, a gzip middleware, or a CDN. Set X-Accel-Buffering: no (or turn buffering off), don't compress the stream, and set Cache-Control: no-cache.

Q5. Your dashboard freezes when a user opens it in seven tabs. Why? SCENARIO

Over HTTP/1.1 the browser allows about six connections per host; six EventSource streams take them all and the seventh request waits. Serve over HTTP/2, or share one stream across tabs (e.g. a SharedWorker).

Q6. EventSource can't set an Authorization header. How do you authenticate SSE? TRICKY

Use cookies (sent automatically, withCredentials for cross-origin), or a short-lived token in the URL, or read the stream with fetch() instead of EventSource, which allows headers and POST. That is how most LLM APIs are consumed.

Q7. Is long polling obsolete? TRICKY

No. It needs nothing special from proxies or firewalls, so it is a reliable fallback; Socket.IO, for example, can start on long polling and upgrade to WebSockets. It is just less efficient per message than SSE.

Next: SD-N-026 · HTTP streaming, chunked transfer and backpressure backendroadmap.com